



FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0070L
(Object-Oriented Programming)

CYCLECARE:
A JAVA CONSOLE-BASED PERSONAL
CYCLE, SYMPTOM, AND SELF-CARE
COMPANION

MIDTERM PROJECT

Prepare by:

Andres, Amanda B. | TN22

Submitted to:

Doc Hadji Tejuco

Submission Date:

October 09, 2026



I. PROJECT TITLE

CycleCare: A Java Console-Based Personal Cycle, Symptom, and Self-Care Companion

II. PROBLEM STATEMENT

“Why does tracking a period sometimes feel like solving a mystery with half the clues missing?”

One forgotten date, one unrecorded symptom, and suddenly it becomes difficult to remember what happened during the previous cycle. For many girls and individuals who menstruate, monitoring period dates, cycle length, cramps, mood changes, fatigue, and energy levels can be challenging. With school requirements, deadlines, responsibilities, and busy routines, these details are often left to memory, making the next period feel unexpected and personal cycle patterns difficult to recognize.

CycleCare was developed to turn these scattered details into organized information. The system allows users to create a personal profile, record period start dates and cycle lengths, save symptoms and wellness details, view previous cycle records, estimate the next period, review basic cycle insights, and receive general food and self-care recommendations based on the selected period day. Through one simple application, users can better understand their cycle without depending entirely on memory or random notes.

However, CycleCare is limited to a Java console-based application. It temporarily stores information only while the program is running and does not include a database, online account, mobile notification system, or cloud storage. Its calculations are calendar-based estimates only and cannot confirm ovulation, diagnose medical conditions, or guarantee pregnancy-related predictions. The recommendations are general wellness suggestions and should not replace professional medical advice.



III. TARGET USERS

CycleCare is primarily designed for girls and individuals who menstruate, especially students and young adults who want a simple way to organize their cycle-related information. It is intended for users who often rely on memory, calendar notes, or scattered information when tracking their periods and symptoms.

The system may be useful for users who want to:

- Record period start dates and cycle lengths;
- monitor symptoms such as cramps, fatigue, and bloating;
- track mood and energy levels;
- view previous cycle records;
- estimate the next period using calendar-based calculations;
- review basic cycle insights; and
- receive general food and self-care recommendations.

CycleCare is particularly suitable for students because it is simple, console-based, and does not require an internet connection or a complicated application interface. It is designed as a personal tracking tool and not as a replacement for professional medical advice.

IV. OBJECTIVES OF THE APPLICATION

Tracking a cycle should feel like a form of self-care, not another task that is easy to forget. With this goal, CycleCare is designed to provide a simple and organized way for users to monitor their menstrual cycle and personal wellness information. Specifically, the application aims to:

1. Create a personal profile where users can enter and view their basic information.
2. Allow users to record multiple period dates and cycle lengths in one organized application.
3. Calculate an estimated next period based on the user's recorded cycle information.
4. Record symptoms, symptom severity, mood, and energy level to help users observe how they feel throughout their cycle.



5. Allow users to view their saved cycle and symptom records instead of relying only on memory.
6. Generate basic cycle insights, including average cycle length, total records, most recent symptoms, and current cycle day.
7. Provide day-based food and self-care recommendations for Days 1 to 5 while recognizing that cycle experiences vary between users.
8. Use input validation to prevent invalid dates, incorrect number entries, empty symptoms, and invalid menu choices.
9. Demonstrate object-oriented programming concepts such as encapsulation, constructors, interfaces, abstraction, inheritance, method overriding, polymorphism, loops, conditions, and object relationships.
10. Provide a beginner-friendly console application that makes cycle tracking more organized, personal, and manageable without requiring an internet connection.

V. SCOPE AND LIMITATIONS

A. Scope

CycleCare is intentionally focused on the everyday side of menstrual cycle tracking: recording information, reviewing personal patterns, and receiving simple wellness suggestions. The application will cover the following features:

- **Profile Management:** Users can enter and view their name, age, and pronouns.
- **Cycle Record Management:** Users can add multiple menstrual cycle records by entering the period start date and cycle length.
- **Cycle Date Estimation:** The system calculates the estimated next period using the recorded period start date and cycle length.
- **Symptom and Wellness Recording:** Users can record the date, symptom, severity level, mood, and energy level connected to their cycle experience.



- **Saved Record Viewing:** Users can view their previously added cycle records and symptom entries while the program is running.
- **Personalized Recommendations:** Users can enter their current period day and receive day-based food and self-care recommendations. Days 1 to 5 have specific suggestions, while other days receive a general recommendation.
- **Cycle Insights:** The system calculates the average cycle length, counts the saved records, displays the most recent symptom, and estimates the current cycle day.
- **Input Validation:** The application checks invalid menu choices, incorrect number inputs, invalid dates, empty names, and empty symptom entries.
- **ObjectOriented Design:** The system uses classes, objects, constructors, private fields, getters, setters, an interface, an abstract class, inheritance, method overriding, polymorphism, loops, conditions, and ArrayList collections.

B. Limitations

Although CycleCare provides useful basic tracking features, it has the following limitations:

- **Console-Based Environment:** CycleCare only operates through a Java console interface and does not include a graphical user interface, mobile application, or online platform.
- **Temporary Data Storage:** The system stores cycle and symptom records in ArrayList objects while the program is running. The information will not be retained after the program is closed.
- **Single Active User:** The current version supports only one user profile during each program session.
- **Manual Data Entry:** Users must manually enter their profile details, period dates, cycle lengths, symptoms, mood, energy level, and selected period day.
- **Calendar-Based Estimates:** The estimated next period and current cycle day are based on basic calendar calculations and may not perfectly represent the user's actual cycle.



- **Predefined Recommendations:** The food and self-care suggestions are predefined according to the selected period day. They do not provide individualized medical, nutritional, or psychological assessments.
- **Unavailable Advanced Features:** The current version does not include database storage, cloud backup, account login, automated notifications, ovulation prediction, fertile-window estimation, or advanced cycle pattern detection.
- **Educational Purpose Only:** CycleCare is intended for personal tracking, general wellness awareness, and educational purposes. It cannot diagnose medical conditions, recommend treatments, or replace professional healthcare advice.

VI. CHATGPT BRAINSTORMING SUMMARY

Summary of AI-Assisted Development Conversation

The development of CycleCare followed six main phases: problem identification, analysis, design, implementation, testing, and improvement. During the problem identification phase, I brainstormed different project ideas related to personal wellness and everyday problems. I eventually chose CycleCare because I wanted to create something relatable and useful for people who have difficulty keeping track of their period dates, symptoms, and wellness information. During the analysis phase, I identified the main features of the system, including profile management, menstrual cycle tracking, symptom recording, day-based food and self-care recommendations, and basic cycle insights.

During the design phase, I identified the classes, attributes, methods, constructors, and relationships that would be needed in the system. I also planned how to apply encapsulation, inheritance, polymorphism, abstraction, and interfaces without making the program unnecessarily complicated. For the implementation phase, I developed CycleCare in Java using NetBeans and created a menu-driven console interface. Different classes were used to handle the user profile, cycle records, symptom entries, recommendations, and cycle analysis.



For the testing phase, I ran the program using different sample inputs to check whether the main features were working properly. I tested the menu, saved records, date calculations, recommendations, cycle insights, and invalid inputs. Some parts were confusing at first, especially when connecting the classes and understanding why the menu returned after every option, but testing helped me understand the program's behavior. During the improvement phase, I identified possible future features such as stronger input validation, more detailed cycle analysis, permanent data storage, and more personalized recommendations. Throughout the process, I used ChatGPT and Gemini as brainstorming partners, programming guides, and code reviewers. They helped me understand Java and OOP concepts whenever I got confused, but I still tested the program myself and decided which suggestions were appropriate for my project.

VII. FINAL PROPOSED SOLUTION

After brainstorming different project ideas, we decided to create CycleCare, a Java console application for menstrual cycle tracking and personal self-care. We chose this idea because it is useful, relatable, and realistic for a student project.

CycleCare allows users to create a profile, record period dates and cycle lengths, save symptoms, mood, and energy levels, view previous records, check cycle insights, and receive food and self-care recommendations based on their selected period day. The system uses classes such as `UserProfile`, `CycleRecord`, `SymptomEntry`, `CycleAnalyzer`, `Recommendation`, `FoodRecommendation`, and `SelfCareRecommendation`. It also demonstrates encapsulation, inheritance, abstraction, interfaces, polymorphism, loops, conditions, constructors, and `ArrayList` collections.

Overall, CycleCare provides a simple way to organize cycle-related information while applying the object-oriented programming concepts required for the project.



VIII. IDENTIFIED CLASSES, ATTRIBUTES, METHODS, AND RESPONSIBILITIES

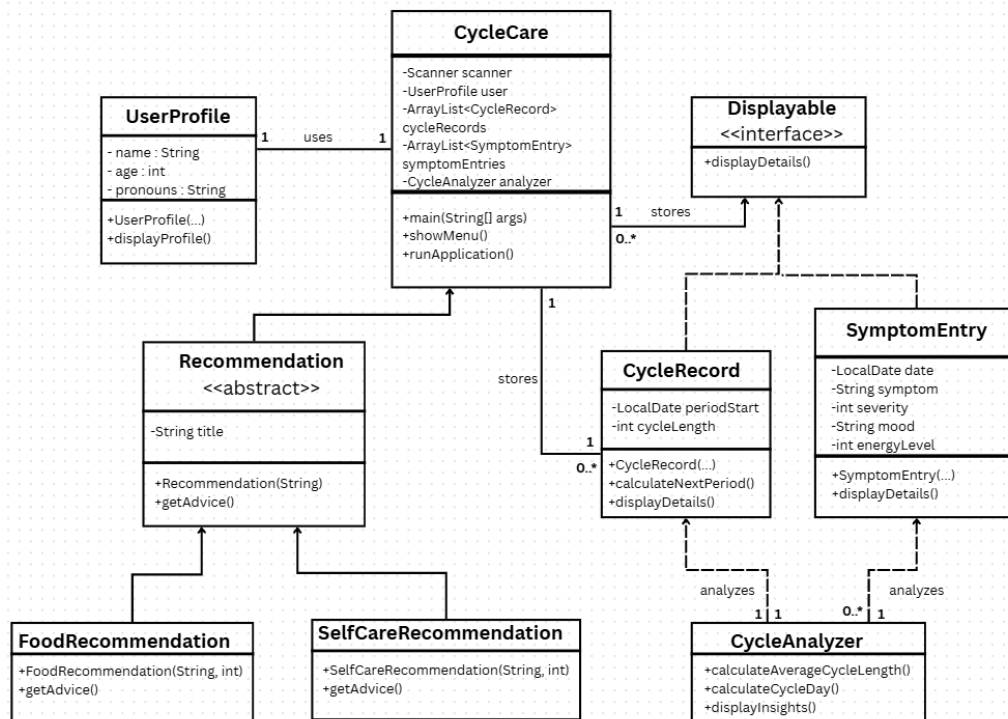
Class	Attributes	Methods	Responsibilities
CycleCare	scanner, user, cycleRecords, symptomEntries, analyzer	main(), createProfile(), runApplication(), showMenu(), addCycleRecord(), viewCycleRecords(), addSymptomEntry(), viewSymptomEntries(), showRecommendations(), viewCycleInsights(), readNumber(), readDate()	Controls the main program, accepts user input, displays the menu, and connects the other classes.
UserProfile	name, age, pronouns	UserProfile(), getters, setters, displayProfile()	Stores and displays the user's personal information.
Displayable	None	displayDetails()	Provides a common method that record classes must implement to display their information.
CycleRecord	periodStart, cycleLength	CycleRecord(), getters, setters, calculateNextPeriod(), displayDetails()	Stores cycle information and calculates the estimated next period.
SymptomEntry	date, symptom, severity, mood, energyLevel	SymptomEntry(), getters, displayDetails()	Stores and displays the user's recorded symptom and wellness details.
Recommendation	title	Recommendation(), getTitle(), getAdvice()	Serves as the abstract parent class for different recommendation types.
FoodRecommendation	periodDay	FoodRecommendation(), getPeriodDay(), getAdvice()	Provides food and hydration recommendation



			s based on the selected period day.
SelfCareRecommendation	periodDay	SelfCareRecommendation(), getPeriodDay(), getAdvice()	Provides rest, comfort, and self-care recommendations based on the selected period day.
CycleAnalyzer	None	calculateAverageCycleLength(), calculateCycleDay(), displayInsights()	Processes cycle and symptom records and displays basic cycle insights.

IX. UML CLASS DIAGRAM

Andres, Amanda B. TN22



CycleCare UML Class Diagram



X. EXPLANATION OF CLASS RELATIONSHIPS

Each class in CycleCare has its own purpose, but they are connected because the features need to work together. We separated the classes so that the main program would not contain all the information and functions in one place.

- **CycleCare and UserProfile.**

The CycleCare class creates one UserProfile object. It collects the user's name, age, and pronouns, then sends these values to the UserProfile constructor. The profile can later be displayed when the user selects the profile option.

- **CycleCare and CycleRecord.**

CycleCare stores the cycle records inside an ArrayList. This allows the user to add more than one

record instead of being limited to only one period entry. Each CycleRecord contains the period start date and cycle length.

- **CycleCare and SymptomEntry.**

CycleCare also stores multiple SymptomEntry objects in another ArrayList. These objects contain the date, symptom, severity, mood, and energy level entered by the user.

- **CycleRecord and Displayable.**

CycleRecord implements the Displayable interface. Because of this, it must have its own displayDetails() method. This method shows the period start date, cycle length, and estimated next period.

- **SymptomEntry and Displayable.**

SymptomEntry also implements Displayable. It has its own displayDetails() method so that the saved symptom information can be shown in a clear format.

- **Recommendation and its subclasses.**

Recommendation is the abstract parent class of FoodRecommendation and SelfCareRecommendation. Both subclasses use extends and call the parent constructor through super(). They also have their own getAdvice() method because the food advice and self-care advice are different.



- **CycleAnalyzer and the record classes.**

CycleAnalyzer uses the saved CycleRecord and SymptomEntry objects to produce basic insights. It calculates the average cycle length, counts the saved records, displays the most recent symptom, and estimates the current cycle day.

Overall, the relationships help keep our program organized. CycleCare handles the main flow, the record classes store user information, the recommendation classes provide advice, and CycleAnalyzer processes the saved records. This makes the program easier to understand and prevents all of the code from being placed in only one class.

XI. EXPLANATION OF ENCAPSULATION

Encapsulation is demonstrated in CycleCare by declaring the attributes in classes such as UserProfile and CycleRecord as private. This prevents other classes from directly changing the data stored inside the objects. Instead, getters and setters are used to access or update the information in a controlled way. For example, the UserProfile class uses methods such as getName(), setName(), getAge(), and setAge(), while the CycleRecord class uses getPeriodStart(), setPeriodStart(), getCycleLength(), and setCycleLength(). This keeps each class responsible for its own data and makes the program more organized, secure, and easier to maintain.

XII. EXPLANATION OF INHERITANCE

Inheritance in CycleCare is shown through the Recommendation class and its two subclasses, FoodRecommendation and SelfCareRecommendation. Recommendation works as the parent class because it contains the common title attribute, constructor, getTitle() method, and abstract getAdvice() method. The two subclasses use extends Recommendation, which allows them to inherit the shared features instead of repeating the same code. They also use super(title) to call the constructor of the parent class. Although both subclasses follow the same basic structure, they provide different advice through their own getAdvice() methods. In this way, inheritance helps the program stay organized while allowing each recommendation class to have its own purpose and personality.



XIII. EXPLANATION OF POLYMORPHISM

Polymorphism is demonstrated in CycleCare when the parent class reference Recommendation is used to hold different subclass objects, specifically FoodRecommendation and SelfCareRecommendation. Even though both variables are declared using the same parent type, each one produces a different result when the getAdvice() method is called because the subclasses provide their own overridden versions of the method. This allows the program to use one common structure for different recommendation types while still displaying advice that matches the object being used. In simple terms, one parent reference can take different forms, which makes the recommendation feature more flexible and easier to expand.

XIV. EXPLANATION OF ABSTRACTION

Abstraction is demonstrated in CycleCare through the abstract Recommendation class. This class contains the common information and structure needed by different recommendation types, such as the title and getAdvice() method, but it does not provide one fixed advice for every situation. Instead, the details of the advice are left to the subclasses, FoodRecommendation and SelfCareRecommendation, which provide their own versions of getAdvice(). This keeps the parent class focused on the general idea of a recommendation while allowing each subclass to handle its specific type of advice. In this way, the program shows only the important structure first and leaves the specific details to the classes that need them.

XV. EXPLANATION OF INTERFACE/S

An interface is demonstrated in CycleCare through the Displayable interface, which contains the displayDetails() method. The interface serves as a common rule for classes that need to show their stored information. Both CycleRecord and SymptomEntry implement Displayable, so they are required to provide their own version of displayDetails(). Although the two classes display different types of information, they follow the same method requirement. This makes the



program more organized because the interface provides a common structure while allowing each class to display its details in its own way.

XVI. IMPORTANT CODE EXCERPTS

A. UserProfile Class

```
public class UserProfile {  
  
    private String name;  
    private int age;  
    private String pronouns;  
  
    public UserProfile(String name, int age, String pronouns) {  
        this.name = name;  
        this.age = age;  
        this.pronouns = pronouns;  
    }  
  
    public void displayProfile() {  
        System.out.println();  
        System.out.println("--- MY PROFILE ---");  
        System.out.println("Name: " + name);  
        System.out.println("Age: " + age);  
        System.out.println("Pronouns: " + pronouns);  
    }  
}
```

- This code shows how UserProfile stores the user's name, age, and pronouns using private attributes. The constructor initializes the values, while displayProfile() displays the saved information in the console.

B. CycleRecord Class

```
private LocalDate periodStart;  
private int cycleLength;  
  
public CycleRecord(LocalDate periodStart, int cycleLength) {  
    this.periodStart = periodStart;  
    this.cycleLength = cycleLength;  
}  
  
public LocalDate getPeriodStart() {  
    return periodStart;  
}
```



```
@Override
public void displayDetails() {
    System.out.println("Period start: " + periodStart);
    System.out.println("Cycle length: " + cycleLength + " days");
    System.out.println("Estimated next period: "
        + calculateNextPeriod());
}
```

- This code shows how CycleRecord stores the period start date and cycle length. It also calculates the estimated next period using plusDays() and displays the cycle details through displayDetails().

C. SymptomEntry Class

```
private LocalDate date;
private String symptom;
private int severity;
private String mood;
private int energyLevel;

public SymptomEntry(LocalDate date, String symptom, int severity,
    String mood, int energyLevel) {
    this.date = date;
    this.symptom = symptom;
    this.severity = severity;
    this.mood = mood;
    this.energyLevel = energyLevel;
}
```

```
@Override
public void displayDetails() {
    System.out.println("Date: " + date);
    System.out.println("Symptom: " + symptom);
    System.out.println("Severity: " + severity + "/10");
    System.out.println("Mood: " + mood);
    System.out.println("Energy level: " + energyLevel + "/10");
}
```



- This code shows how SymptomEntry stores the date, symptom, severity, mood, and energy level entered by the user. The constructor initializes the values, while displayDetails() shows the saved symptom information.

D. Recommendation and Subclasses

- Recommendation.java:

```
public abstract class Recommendation {  
  
    private String title;  
  
    public Recommendation(String title) {  
        this.title = title;  
    }  
  
    public String getTitle() {  
        return title;  
    }  
  
    public abstract String getAdvice();  
}
```

- This code shows Recommendation as an abstract parent class. It provides the common title and getAdvice() structure that the recommendation subclasses will use.

- FoodRecommendation.java

```
public class FoodRecommendation extends Recommendation {  
  
    private int periodDay;  
  
    public FoodRecommendation(String title, int periodDay) {  
        super(title);  
        this.periodDay = periodDay;  
    }  
  
    default:  
        return "Choose balanced meals, drink enough water, "  
            + "and listen to what your body needs.";  
    }  
}
```



```
@Override
public String getAdvice() {
    switch (periodDay) {
        case 1:
            return "Day 1: Drink enough water and choose warm, "
                + "balanced meals if that feels comfortable.";
    }
}
```

- This code shows FoodRecommendation inheriting from Recommendation through extends. It uses super(title) to call the parent constructor and overrides getAdvice() to provide food recommendations based on the selected period day.

- SelfCareRecommendation.java:

```
public class SelfCareRecommendation extends Recommendation {

    private int periodDay;

    public SelfCareRecommendation(String title, int periodDay) {
        super(title);
        this.periodDay = periodDay;
    }

    @Override
    public String getAdvice() {
        switch (periodDay) {
            case 1:
                return "Day 1: Use a warm compress if you have cramps, "
                    + "stay hydrated, and rest when needed.";

            default:
                return "Listen to your body and follow the self-care habits "
                    + "that feel comfortable for you.";
        }
    }
}
```

- This code shows SelfCareRecommendation using the same parent class but providing its own self-care advice. The overridden getAdvice() method allows this subclass to behave differently from FoodRecommendation.

E. ArrayList and Menu Implementation

```
private ArrayList<CycleRecord> cycleRecords;
private ArrayList<SymptomEntry> symptomEntries;
```

- These ArrayList attributes allow the program to store multiple cycle records and symptom entries instead of limiting the user to only one record.



```
cycleRecords = new ArrayList<>();  
symptomEntries = new ArrayList<>();
```

- These lines add the newly created CycleRecord and SymptomEntry objects into their respective ArrayLists so they can be viewed later.

```
choice = readNumber("Enter your choice: ", 0, 7);  
  
switch (choice) {  
    case 1:  
        user.displayProfile();  
        break;  
  
    case 2:  
        addCycleRecord();  
        break;  
  
    case 3:  
        viewCycleRecords();  
        break;  
  
    case 4:  
        addSymptomEntry();  
        break;  
  
    case 5:  
        viewSymptomEntries();  
        break;  
  
    case 6:  
        showRecommendations();  
        break;  
  
    case 7:  
        viewCycleInsights();  
        break;  
  
    case 0:  
        System.out.println();  
        System.out.println("Thank you for using CycleCare.");  
        System.out.println(  
            "Take care of yourself, one day at a time.");  
        break;
```

- This code controls the main menu of CycleCare. The switch statement checks the user's selected option and opens the correct feature. The loop surrounding the menu allows the program to continue running until the user chooses 0 to exit.



F. CycleAnalyzer Class

```
public double calculateAverageCycleLength(  
    ArrayList<CycleRecord> records) {  
  
    if (records.isEmpty()) {  
        return 0;  
    }  
  
    int total = 0;  
  
    for (CycleRecord record : records) {  
        total += record.getCycleLength();  
    }  
  
    return (double) total / records.size();  
}
```

- This method calculates the average cycle length by adding the cycle lengths stored in the ArrayList and dividing the total by the number of records. It also checks if there are no records before performing the calculation.

XVII. SCREENSHOTS OF PROGRAM EXECUTION

A. Profile Creation

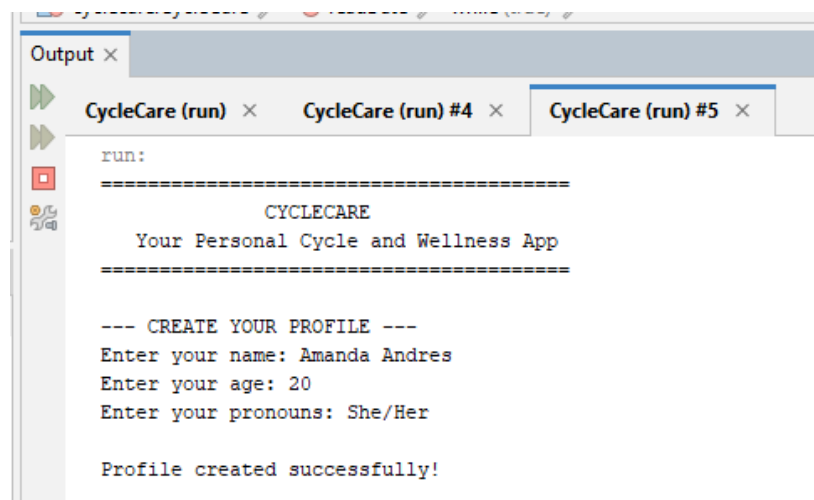


Figure 1. Profile creation and user input



B. Main Menu

```
=====
                        CYCLECARE
=====
[1] View My Profile
[2] Add Cycle Record
[3] View Cycle Records
[4] Add Symptom Entry
[5] View Symptom Entries
[6] Get Personalized Recommendations
[7] View Cycle Insights
[0] Exit
=====
Enter your choice:
```

Figure 2. CycleCare main menu

C. Adding a Cycle Record

```
Enter your choice: 2

--- ADD CYCLE RECORD ---
Enter period start date (YYYY-MM-DD): 2026-10-02
Enter cycle length in days: 28

Cycle record added successfully!

=====
                        CYCLECARE
=====
```

Figure 3. Adding a cycle record

D. Viewing Cycle Records

```
--- CYCLE RECORDS ---

Record 1
Period start: 2026-10-02
Cycle length: 28 days
Estimated next period: 2026-10-30
```

Figure 4. Viewing saved cycle records



E. Adding a Symptom Entry

```
--- ADD SYMPTOM ENTRY ---  
Enter symptom date (YYYY-MM-DD): 2026-10-02  
Enter symptom: cramps  
Enter symptom severity from 1 to 10: 8  
Enter mood: tired  
Enter energy level from 1 to 10: 10  
  
Symptom entry added successfully!
```

Figure 5. Adding a symptom entry

F. Personalized Recommendations

```
--- PERSONALIZED RECOMMENDATIONS ---  
Enter your current period day: 1  
  
Period day: 1  
  
Food and hydration:  
Day 1: Drink enough water and choose warm, balanced meals if that feels comfortable.  
  
Rest and comfort:  
Day 1: Use a warm compress if you have cramps, stay hydrated, and rest when needed.  
  
These are general wellness suggestions only.
```

Figure 6. Day-based personalized recommendations

G. Cycle Insights

```
--- CYCLE INSIGHTS ---  
Average cycle length: 28.0 days  
Total cycle records: 1  
Total symptom entries: 1  
Most recent symptom: cramps  
Current cycle day: Day 8
```

Figure 7. Cycle insights and calculations



H. Program Exit

```
Thank you for using CycleCare.  
Take care of yourself, one day at a time.  
BUILD SUCCESSFUL (total time: 9 minutes 32 seconds)
```

Figure 8. Successful program exit

XVIII. TEST CASE TABLE

Test #	Feature	Input	Expected Result	Actual Result	Pass/Fail
1	Valid Profile	Amanda Andres, age 20, she/her	Profile is created and the main menu appears.	Profile created and main menu displayed.	Pass
2	Minimum Age	Age: 1	Age is accepted because it is the minimum allowed value.	Age accepted.	Pass
3	Maximum Reasonable Age	Age: 120	Age is accepted because it is the maximum allowed value.	Age accepted.	Pass
4	Invalid Age	Age: 0	Age is rejected and the program asks for a value from 1 to 120.	Error message displayed and input requested again.	Pass
5	Minimum Cycle Length	Cycle length: 1	Cycle record is saved successfully.	Cycle record saved.	Pass
6	Maximum Cycle Length	Cycle length: 60	Cycle record is saved successfully.	Cycle record saved.	Pass
7	Invalid Date	2026/10/02	Date is rejected and the program asks for YYYY-MM-DD format.	Invalid date format. Please use YYYY-MM-DD.	Pass
8	Empty Symptom	Leave the symptom input blank	Empty input is rejected and the program asks for a symptom.	Symptom cannot be empty. Enter symptom:	Pass
9	Incorrect Menu Selection	Enter 9	Menu choice is rejected and the program asks for	Please enter a number from 0	Pass



			a number from 0 to 7.	to 7. Enter your choice:	
10	Repeated Cycle Records	Add two different cycle records, then select option 3.	Both cycle records are saved and displayed.	Both records displayed.	Pass
11	Symptom Entry Object	Enter cramps, severity 8, mood tired, and energy 10.	SymptomEntry object is created, saved, and displayed.	Symptom entry displayed successfully.	Pass
12	Different Recommendation Subclasses	Select option 6 and enter period days 1 and 3.	FoodRecommendation and SelfCareRecommendation display different advice based on the selected day.	Both recommendation types displayed with day-based advice.	Pass

XIX. PROBLEMS ENCOUNTERED DURING DEVELOPMENT

- **Choosing the Right Classes** - At the beginning, I was confused about which classes CycleCare actually needed. I had many ideas for the system, so it was difficult to decide which features were important and which ones would only make the project more complicated. I eventually focused on UserProfile, CycleRecord, SymptomEntry, Recommendation, its subclasses, and CycleAnalyzer because each class had a clear purpose.
- **Understanding Interfaces and OOP Concepts** - I also had difficulty understanding where to use the Displayable interface, inheritance, abstraction, and polymorphism. At first, I felt like I was only adding concepts because they were required. After connecting the interface to CycleRecord and SymptomEntry, and using Recommendation as the parent class, I understood how these concepts could actually fit the system.
- **Understanding the Repeating Menu** - While testing the View Cycle Records option, I initially thought the program was not working because it returned to Enter your choice



after displaying the record. I later realized that the do-while loop was working correctly. It was designed to return to the main menu so the user could choose another feature instead of ending the program immediately.

- **Connecting the Recommendation Classes** - I found it challenging to connect FoodRecommendation and SelfCareRecommendation to the main program. I had to understand how the Recommendation class, super(), method overriding, and polymorphism worked together. After adjusting the showRecommendations() method, the program was able to display different food and self-care suggestions based on the selected period day.
- **Managing Multiple Records** - At first, the program could only work with one cycle record and one symptom entry at a time. I needed to use ArrayList so users could add and view multiple records. This was confusing at first, but I eventually understood that the ArrayList acts as a container for the CycleRecord and SymptomEntry objects.
- **Testing Invalid Inputs** - Another challenge was making sure that the program would not immediately crash when the user entered an invalid date, number, blank symptom, or incorrect menu choice. I used loops, conditions, and try-catch statements so the program could display an error message and ask the user to enter the information again.
- **Understanding the Cycle Calculations** - I also needed to understand how the program calculated the estimated next period, average cycle length, and current cycle day. Working with LocalDate, ArrayList, loops, and date calculations was new to me, so I tested the program using different dates and cycle lengths until I understood how the results were produced.

XX. HOW CHATGPT HELPED

I used ChatGPT and Gemini as guides while I was making CycleCare. I did not use them to completely make the project for me. I mostly asked for help whenever I got confused about what to do next, especially when I was connecting the classes, creating the menu, using ArrayList, and applying the OOP concepts required for the project. Since I was refreshing my Java knowledge



after using C++ before, I also used them to understand the differences in syntax and how certain concepts worked in Java.

I also sent my code and console outputs so I could check if what I made was working properly. ChatGPT helped me understand why the program kept returning to the menu, how the recommendation classes were connected, and how the cycle calculations worked. I still tested the code myself in NetBeans

and decided which suggestions actually made sense for my project. Overall, I used ChatGPT and Gemini more like study partners and code reviewers whenever I needed clarification, not as replacements for my own work.

XXI. AI SUGGESTIONS ACCEPTED

During the development of CycleCare, I used ChatGPT and Gemini to ask for ideas, explanations, and feedback. I accepted suggestions that were useful, understandable, and connected to the requirements of the project.

- AI suggestion: ChatGPT suggested creating a menstrual cycle and wellness tracking system instead of making a general personal productivity application.
 - My evaluation: I found the idea more useful and relatable because it focused on one specific problem while still having enough features for an Object-Oriented Programming project.
 - My decision: I accepted the suggestion and named the system CycleCare.
- AI suggestion: ChatGPT suggested using a Recommendation parent class with FoodRecommendation and SelfCareRecommendation as subclasses.
 - My evaluation: I accepted this because both classes are types of recommendations, which makes their relationship an appropriate “is-a” relationship. They also share a title and getAdvice() method, while still providing different types of advice.



- My decision: I accepted the suggestion and used Recommendation as an abstract class. FoodRecommendation and SelfCareRecommendation extend it and override the getAdvice() method.
- AI suggestion: ChatGPT suggested using ArrayList to store multiple CycleRecord and SymptomEntry objects.
 - My evaluation: This made sense because the program should not be limited to only one cycle record or one symptom entry.
 - My decision: I accepted the suggestion and used separate ArrayList objects for cycle records and symptom entries.
- AI suggestion: ChatGPT suggested adding input validation for dates, numbers, empty symptoms, and menu choices.
 - My evaluation: I accepted this because the program should not immediately crash when the user enters incorrect information.
 - My decision: I added loops, conditions, and try-catch statements so the program can display an error message and ask for the input again.

XXII. AI SUGGESTIONS REJECTED OR MODIFIED

I did not accept every suggestion given by ChatGPT and Gemini. I rejected or changed suggestions that were too complicated, unnecessary, or not suitable for the current version of my project.

- AI suggestion: ChatGPT initially suggested using general class names such as User and PeriodRecord.
 - My evaluation: I felt that these names were too general and did not clearly describe the purpose of the classes in CycleCare.
 - My decision: I modified the suggestion by using UserProfile and CycleRecord because these names fit my system better and were easier for me to understand.
- AI suggestion: ChatGPT suggested adding many features such as journaling, event planning, fertility estimation, PMS pattern detection, and permanent data storage



- My evaluation: I thought these features were interesting, but including all of them would make the project too large and complicated for the time and requirements given.
- My decision: I rejected or deferred these features and focused on profile management, cycle tracking, symptom monitoring, recommendations, and basic cycle insights.
- AI suggestion: ChatGPT suggested adding validation directly to some class setters even though the main program already validates user input.
 - My evaluation: I understood the purpose of the suggestion, but I felt that adding it immediately would make my working code more complicated than necessary.
 - My decision: I decided to keep my original working implementation and use the validation methods in the main program.
- AI suggestion: ChatGPT suggested adding ovulation prediction and fertile-window estimation.
 - My evaluation: I decided that these features could produce misleading results because menstrual cycles are different for every person.
 - My decision: I deferred these features and kept the program focused on general calendar-based estimates and wellness suggestions.

XXIII. WHAT I PERSONALLY LEARNED

I made CycleCare while also studying for my other midterm exams, so honestly, my brain was already tired before I even started coding. There were moments when I wanted to stop because I still had other subjects to review, but I also enjoyed working on it because I knew that this kind of application could actually be useful to girls like me who always lose track of their period dates, symptoms, and moods. It felt different from making a random system because I could actually relate to the problem.



This project also made me realize that coding is really hard. It is not just about typing code and hoping that it works. The program is very logical, so it only does exactly what I tell it to do, even when what I told it to do is wrong. I spent hours finishing CycleCare because I had to make sure that the classes, objects, methods, menu, and inputs were connected properly. One small mistake could already affect another part of the program.

I also understood the codes and OOP concepts better while creating the system. I learned that `UserProfile` stores the user's information, `CycleRecord` handles the cycle dates, and `SymptomEntry` stores the symptoms and wellness details. I learned how `ArrayList` can store multiple objects, how the loop keeps the menu running, and how the switch statement controls the user's choices. The `Recommendation`, `FoodRecommendation`, and `SelfCareRecommendation` classes also helped me understand inheritance, abstraction, overriding, and polymorphism because the recommendations could change depending on the selected period day.

I am thankful for my friends, ChatGPT, and Gemini because they helped me whenever I got confused or forgot how something worked in Java. I still had to test the code myself and decide which suggestions

I wanted to use. Overall, this project taught me that coding takes a lot of patience, logic, and trial and error. It was stressful doing this while preparing for other midterms, but finishing CycleCare made me realize that I can create something useful even when I feel like my brain already needs its own restart button.

XXIV. CONCLUSION

What started as a midterm requirement turned into a small application with a purpose that feels much bigger than a grade. CycleCare was developed to help users organize their menstrual cycle information, symptoms, mood, energy levels, and basic wellness habits in one place. Through its profile management, cycle records, symptom entries, day-based recommendations,



and cycle insights, the system provides a simple way to make period tracking less confusing and less dependent on memory.

Developing CycleCare also allowed the required Object-Oriented Programming concepts to become part of an actual working system. The project used classes, objects, constructors, private fields, getters, setters, interfaces, abstraction, inheritance, method overriding, polymorphism, ArrayList, loops, conditions, menu navigation, and date calculations. Each concept had a purpose in the program, which made it easier to understand how the different parts of Java work together instead of studying them as separate definitions.

Although CycleCare is still limited to a console application and only stores information while the program is running, it successfully demonstrates how a simple idea can be turned into a functional system. More advanced features such as permanent storage, notifications, record editing, and more detailed cycle analysis can be added in the future. Most importantly, this project showed me that coding is not only about making a program run. It is also about creating something useful, solving a problem that people can relate to, and learning how to turn an idea into something real, one line of code at a time.